Solution Architecture

Date	17 September 2026
Team ID	SWTID-2026-2630
Project Name	AI Fit Track API
Maximum Marks	4 Marks

Solution Architecture – Layered Component View

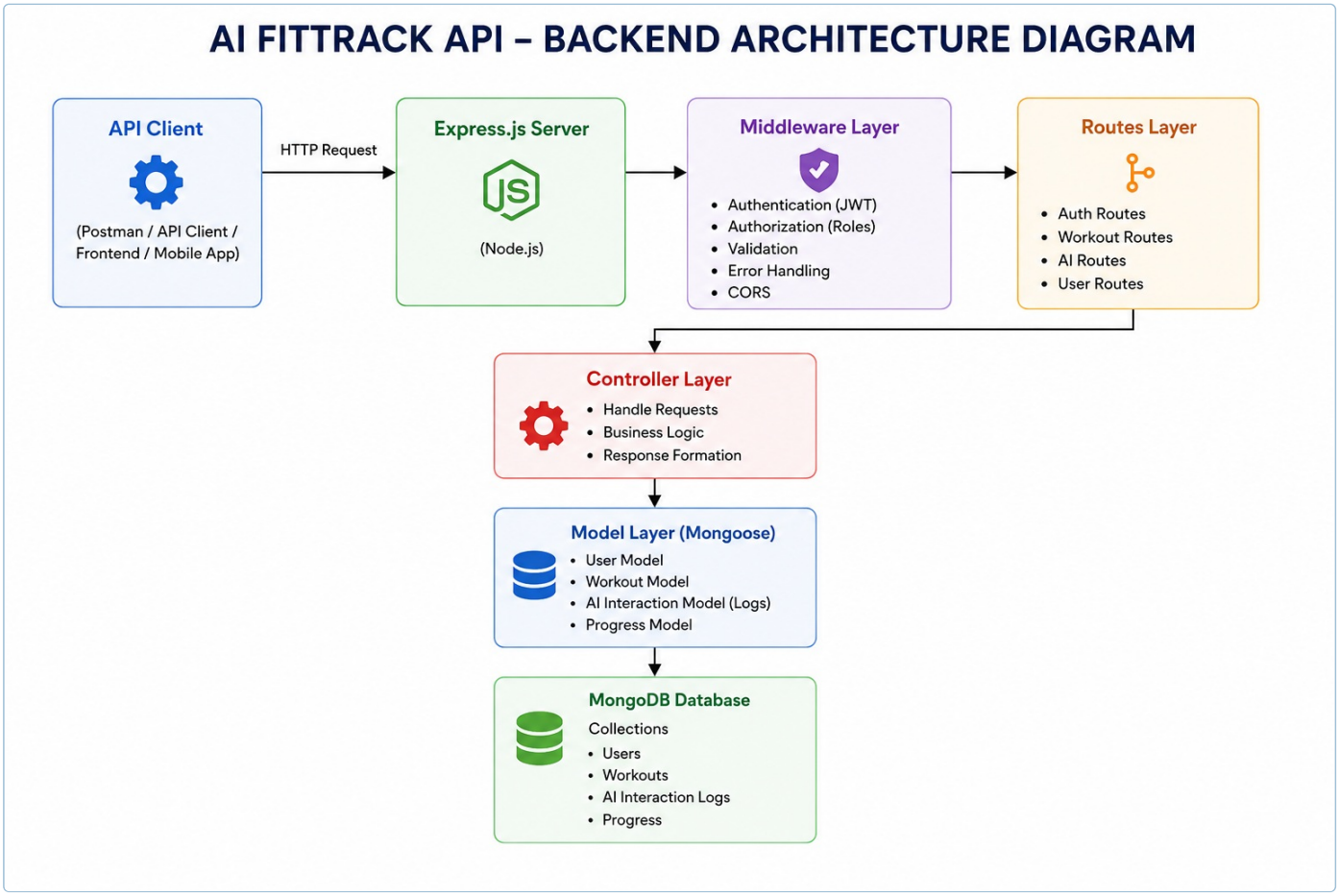


Fig. 1 – AI Fit Track layered architecture, as designed by the team

Component Description

Layer	Responsibility
Client Layer	Thunder Client / Postman during development and grading (a `client/` folder is reserved for a future React/mobile UI); sends HTTP requests with JSON bodies and, where required, a Bearer JWT.
Express Server / Gateway	`server.js` (or `app.js`) bootstraps Express, applies `cors` and JSON body-parsing middleware, and mounts all route groups.
Authentication Middleware	`middleware/auth.js` reads the `Authorization: Bearer <token>` header, verifies the JWT with `jsonwebtoken`, and attaches the decoded user to `req.user` before allowing the request onward.
Route Layer	`routes/authRoutes.js`, `routes/workoutRoutes.js`, `routes/aiRoutes.js` declare each endpoint's path, HTTP method, and which middleware/controller handles it.
Controller Layer	`controllers/authController.js`, `workoutController.js`, `aiController.js` hold request-handling logic: input validation, calling models/services, and shaping the JSON response.
Service Layer	`services/geminiService.js` isolates all Google Gemini AI calls (prompt construction, response parsing) so controllers stay free of AI-vendor-specific code.
Database Layer	`models/User.js` and `models/Workout.js` (Mongoose schemas) plus `config/db.js` (MongoDB connection) persist and validate all data.

MVC Design Pattern

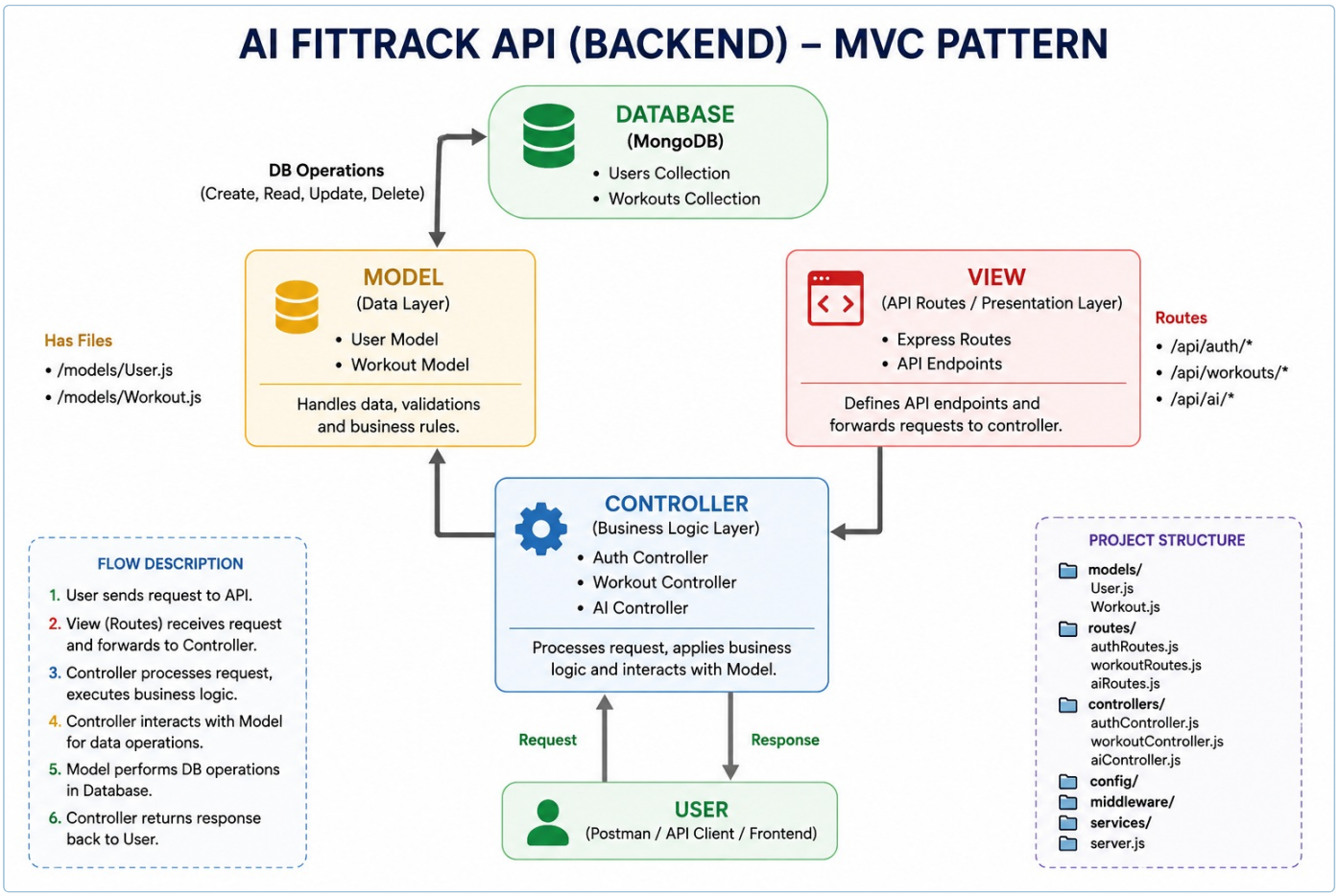


Fig. 2 – MVC mapping used across the codebase

Models (`User`, `Workout`) define schema and validation; Controllers contain the request/response logic and orchestrate calls to models and the Gemini service; there is no server-rendered View layer since the project is a pure JSON REST API – the "view" is the JSON response itself, consumed by whichever client (Thunder Client, Postman, or a future frontend) issues the request.